

Tema 6. Memoria caché

Configuración y funcionamiento

Arquitectura de computadores - Semana 12

Grado en *Ingeniería Informática*

Departamento de Informática. Universidad de Jaén



Objetivos

General

Comprender el impacto que tiene la memoria caché en el rendimiento de un procesador a través del estudio de su configuración y funcionamiento

Específicos

- **Razonar** la necesidad de contar con una memoria caché
- **Saber** el lugar que ocupa la memoria caché en la arquitectura de un ordenador
- **Diferenciar** entre la caché de instrucciones y la de datos
- **Conocer** el funcionamiento de una memoria caché
- **Identificar** las distintas configuraciones de una memoria caché
- **Analizar** el impacto de la memoria caché en el rendimiento general

**¿Por qué necesitamos
una memoria caché?**



CPU y memoria RAM

Funcionamiento

La CPU precisa traer a su cauce de ejecución las instrucciones del programa, así como leer y escribir desde la memoria RAM los datos sobre los que debe operar

Interacción CPU $\leftrightarrow$ RAM

- **Instrucciones:** la CPU precisa captar las instrucciones del programa que ejecuta de manera continua
- **Datos:** en el mismo ciclo de reloj pueden precisarse múltiples accesos de lectura y/o escritura en memoria
- **Segmentación:** los procesadores segmentados incrementan la demanda de accesos a memoria, de forma que existen varios en cada ciclo de reloj
- **Superescalares:** una arquitectura superescalar precisa captar varias instrucciones por ciclo y ejecutar múltiples accesos a datos

CPU vs memoria RAM

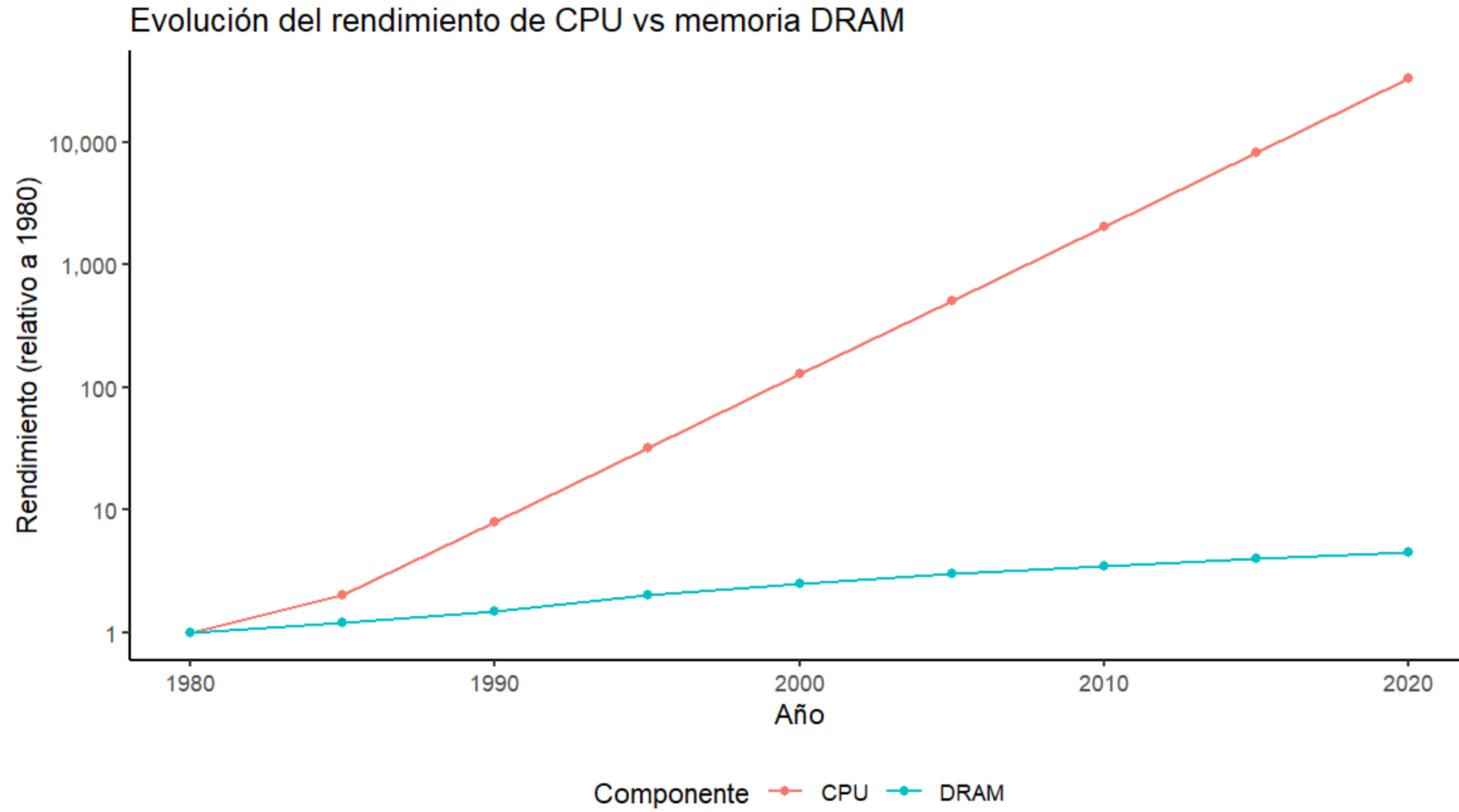
Velocidad de funcionamiento

Una CPU moderna opera a una velocidad varios órdenes de magnitud más alta que la memoria DRAM estándar, por lo que esta se convierte en un cuello de botella

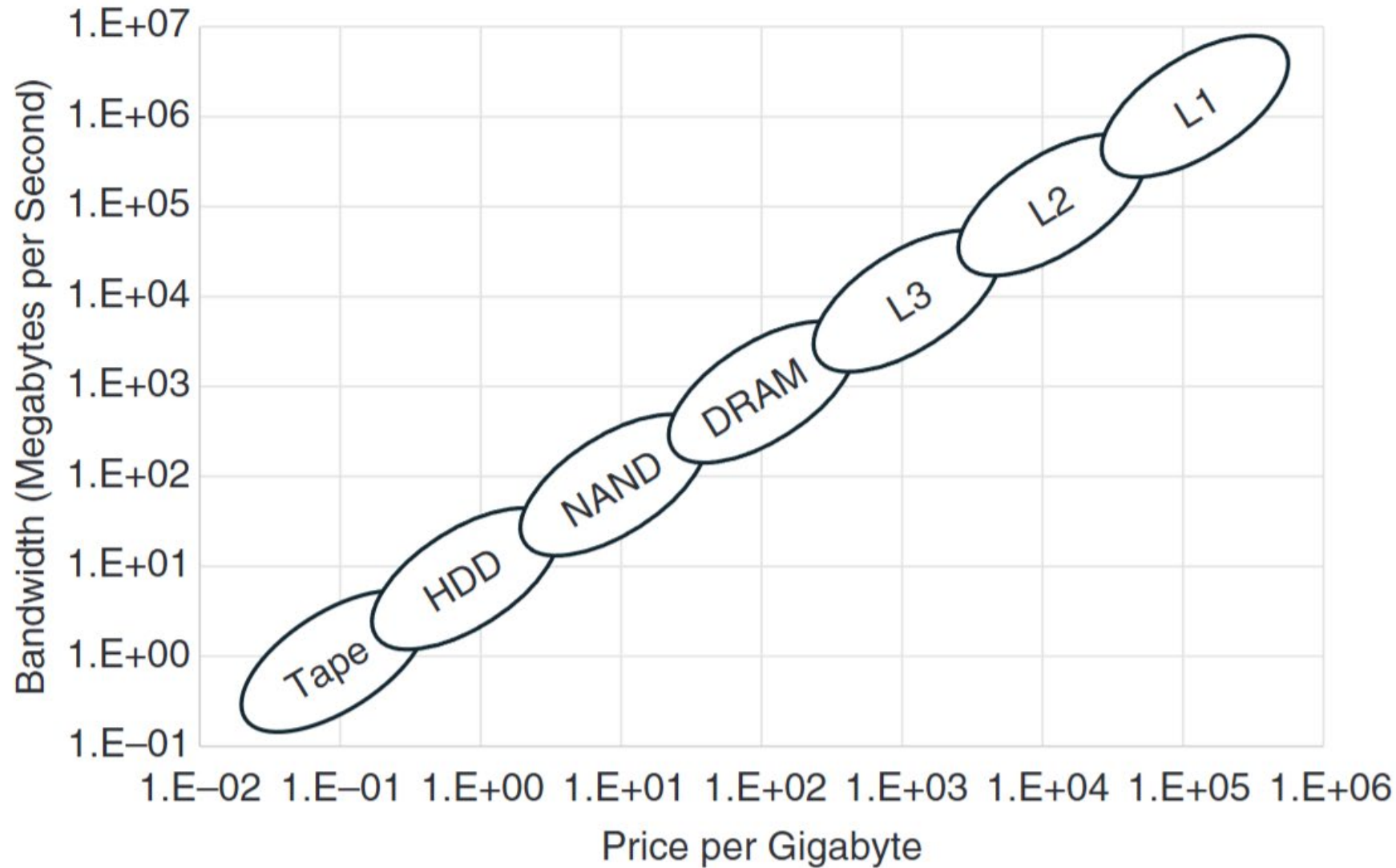
Consecuencias y soluciones

- **Latencia:** la lectura/escritura en el banco de registros es inmediata (< 1 ciclo), mientras que el tiempo de espera desde que se emite una orden de lectura/escritura hasta que la memoria RAM responde es > 100 ciclos (según CPU y su frecuencia)
- **Detenciones:** un acceso a memoria provocaría que la CPU detuviese el cauce de ejecución durante un tiempo muy significativo, de forma que las mejoras de la segmentación y arquitectura superescalar pasarían desapercibidas
- **Memoria caché:** la solución estriba en situar cerca de la CPU una memoria, mucho más rápida que la RAM, para reducir la latencia y evitar las detenciones. Esta es la que denominamos memoria caché

Diferencial de velocidad CPU vs RAM



Coste vs ancho de banda de las memorias



"The future of data", noviembre 2025, *IEEE Computer*

Situación de la caché en la arquitectura



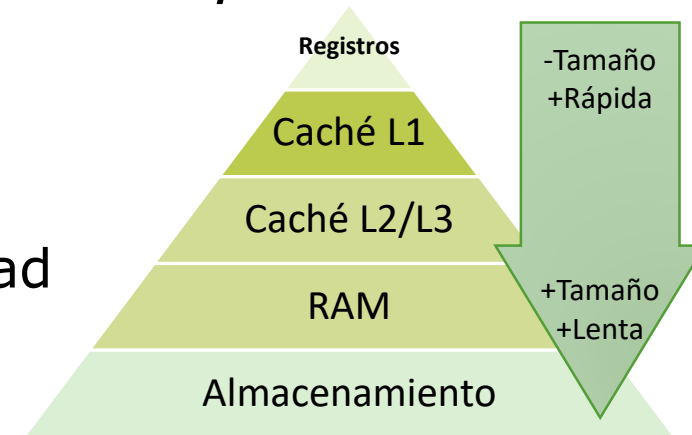
Niveles de memoria

La memoria en un computador

En la arquitectura de un computador existen diferentes niveles de memoria, cuyas características cambian según su situación respecto al procesador

Características de cada tipo de memoria

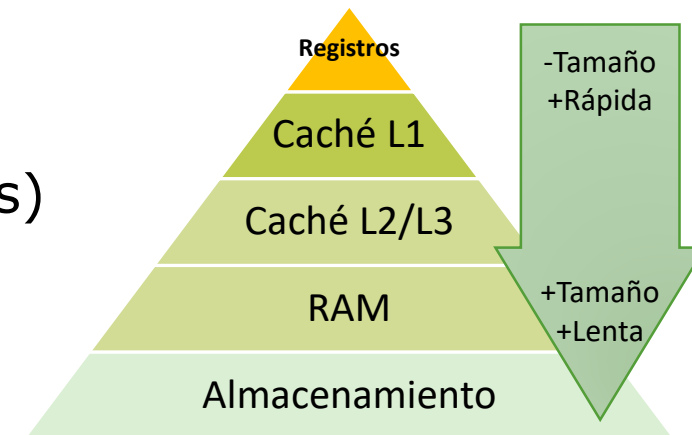
- **Registros:** internos al procesador, poca capacidad, alta velocidad
- **Caché L1:** cercana al procesador, más capacidad que los registros, casi igual de rápida
- **Caché L2/L3:** menos cercanía a la CPU, mayor tamaño que la L1 pero más lenta
- **RAM:** externa al procesador, mucha más capacidad pero menor velocidad. Comunicación por bus rápido dedicado
- **Almacenamiento:** discos de estado solido o mecánicos. Gran tamaño pero muy lentos en comparación. Comunicación por un bus lento genérico



Niveles de memoria

Registros internos

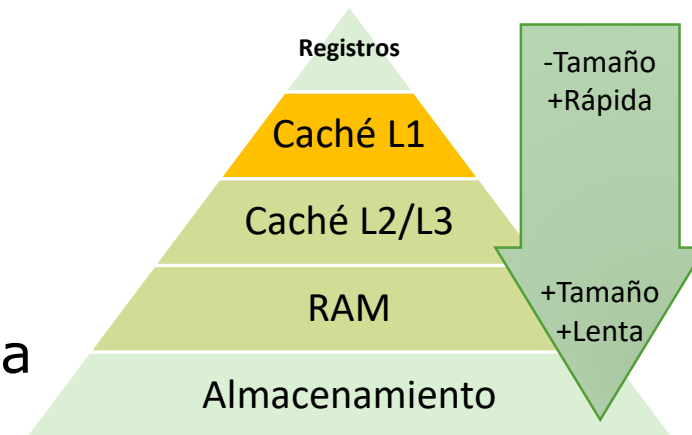
- **Naturaleza.** Pequeñas porciones de memoria, habitualmente 32 o 64 bits por registro, en los que la CPU opera de manera directa
- **Tamaño.** Los procesadores cuentan con un banco de registros, más o menos grande, y acceden a ellos de manera individual a través de un bus de direcciones específico y reducido (p.e. 5 bits)
- **Velocidad.** La lectura y escritura en un registro de memoria se realiza durante el procesamiento de las instrucciones y emplea una fracción de ciclo del procesador
- **Tecnología.** Los registros se crean con la misma tecnología de fabricación de la CPU (habitualmente CMOS) y operan a la misma velocidad
- **Comunicación.** El contenido de los registros se lee/escribe a través de las líneas internas que conectan el banco de registros con la ALU y otros elementos de la CPU



Niveles de memoria

Nivel 1 de memoria caché

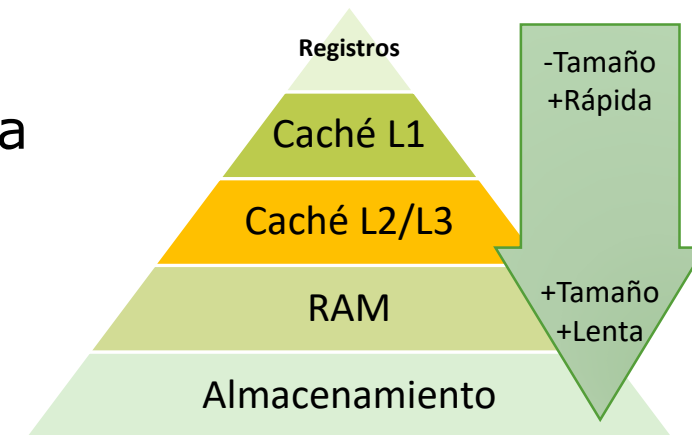
- **Naturaleza.** Este nivel de caché se divide en dos porciones específicas, una para instrucciones y otra para datos, y forma parte del circuito integrado junto a la CPU
- **Tamaño.** Es una memoria de poca capacidad, lo habitual son entre 8 y 64 KB según el procesador
- **Velocidad.** La lectura y escritura en esta memoria se realiza típicamente en un ciclo de procesador si la información requerida está almacenada en la caché
- **Tecnología.** La memoria caché se crea con tecnología SRAM, por lo que no precisa refresco pero resulta muy cara, y la escala de integración es la misma que la de la CPU
- **Comunicación.** La CPU emplea técnicas de comunicación específicas para acceder a la caché L1 (p.e. el decodificador de la CPU está conectado directamente a la caché de instrucciones)



Niveles de memoria

Nivel 2/3 de memoria caché

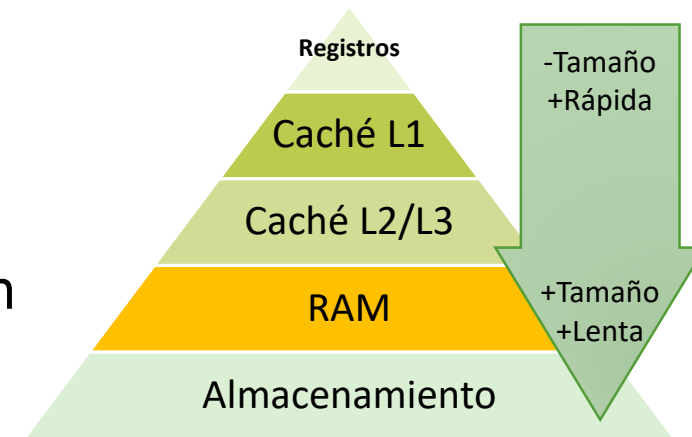
- **Naturaleza.** Uno o dos niveles adicionales de caché que, aunque pueden estar en el mismo circuito integrado que la CPU, no forman parte de esta. Almacenan tanto código como datos
- **Tamaño.** Son memorias con una capacidad mucho mayor que la caché L1, del orden de varios megabytes en los procesadores actuales
- **Velocidad.** El acceso a la L2/L3 no es tan rápido como a la L1, requiere varios ciclos de procesador. La L2 es más cercana y rápida que la L3
- **Comunicación.** Mientras que cada CPU en un circuito integrado tiene su propia caché L1, la caché L2/L3 es compartida por todas las CPU (*cores*) y precisa un bus de comunicación más complejo. Si existen L2 y L3, la primera será habitualmente local a cada núcleo y la segunda compartida



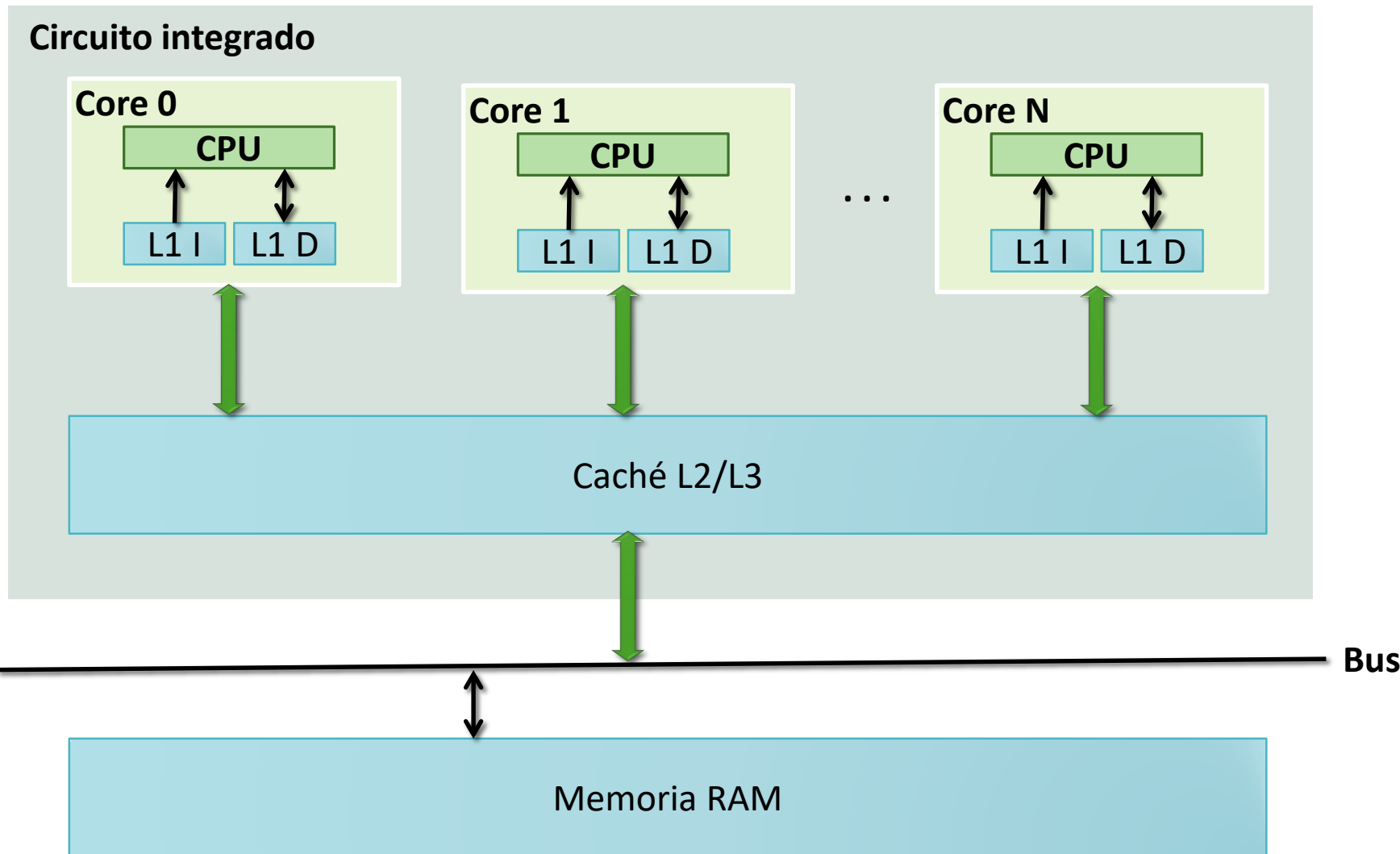
Niveles de memoria

Memoria RAM

- **Naturaleza.** Es una memoria externa al circuito integrado donde se aloja la CPU, pensada para alojar tanto código como datos durante la ejecución del software
- **Tamaño.** La capacidad de este tipo de memoria se mide en la actualidad en GB (gigabytes)
- **Velocidad.** Las operaciones de lectura/escritura en RAM pueden llegar a ser dos órdenes de magnitud más lentas que en caché L1, ya que su frecuencia de funcionamiento no es tan alta
- **Tecnología.** La escala de integración para memorias RAM no suele ser tan alta como para los microprocesadores
- **Comunicación.** Los buses de direcciones y datos de la CPU se conectan a esta memoria, para seleccionar una celdilla y leer o escribir en ella, pero son buses compartidos entre todos los núcleos y otros dispositivos de acceso a memoria



Ubicación de la caché



Elemento	Tecnología	Tamaño	Ciclos
Registros	CMOS	32 X 64bits	~ 1
Caché L1	SRAM	8-64 KB	~ 4
Caché L2	SRAM	> 256 KB	~ 12
Caché L3	SRAM	> 4 MB	~ 50
RAM	DRAM	> 1 GB	> 100

Detalles sobre tamaño y latencia de memorias caché de distintos procesadores en 7-cpu.com

Cachés de instrucciones y de datos



Separación en cachés de instrucciones y datos

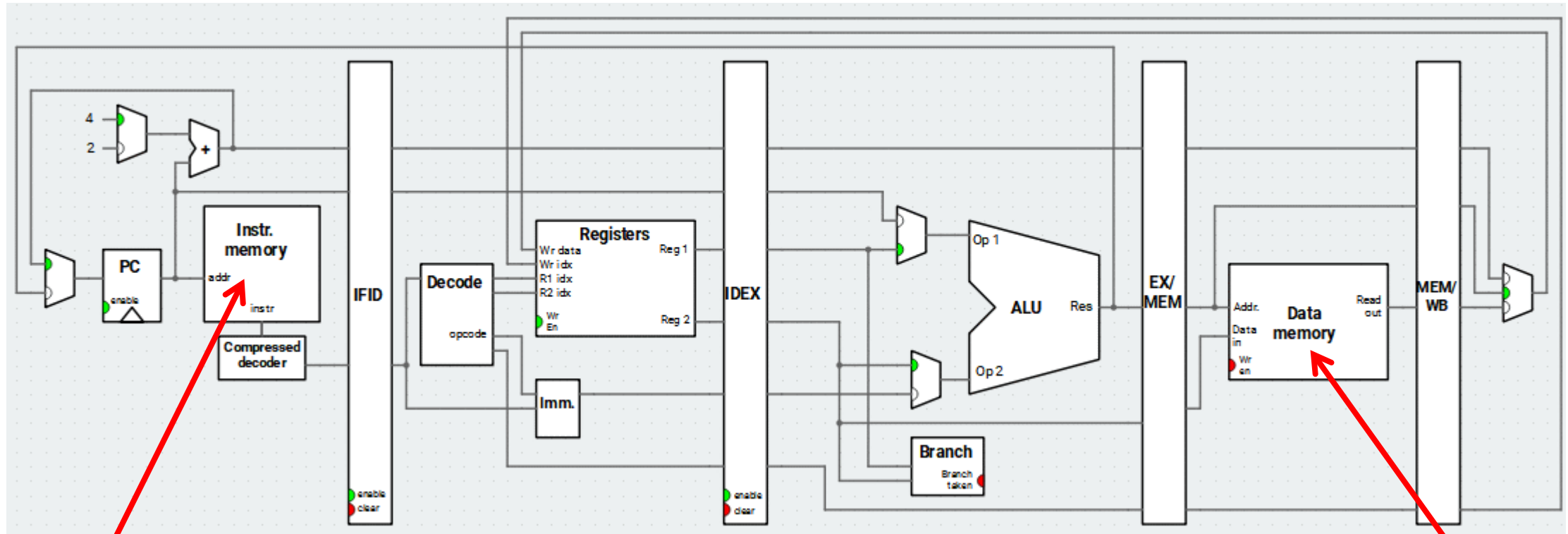
Razonamiento

La separación hace posible el acceso simultáneo a ambas cachés, de forma que pueden captarse instrucciones y leerse/escribirse datos en memoria en el mismo ciclo de reloj

Descripción

- **Niveles inferiores de memoria:** tanto la memoria RAM como las cachés L2 y L3 se ajustan a la arquitectura Von Neumann, con una estructura unificada que almacena tanto el programa como los datos sobre los que este opera
- **Separación:** cuando el contenido de la memoria RAM se lleva a la caché L1 se separan instrucciones y datos, dando lugar a una arquitectura Harvard más eficiente
- **L1I:** la caché L1 de instrucciones es de solo lectura y los patrones de acceso son fácilmente predecibles
- **L1D:** la caché L1 de datos es de lectura y escritura, con patrones de acceso menos predecibles

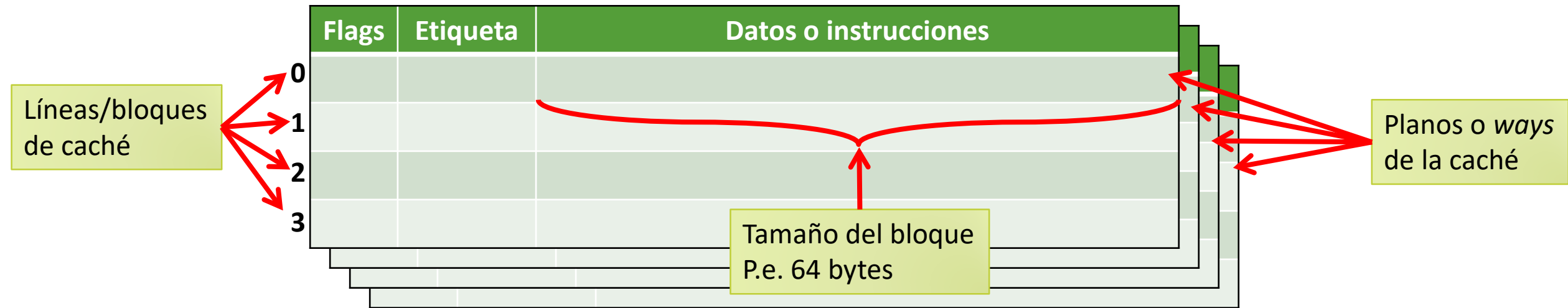
Cachés L1I y L1D en el cauce de ejecución



L1I - Cache L1
instrucciones

L1D - Cache L1
datos

Estructura de las cachés L1I y L1D



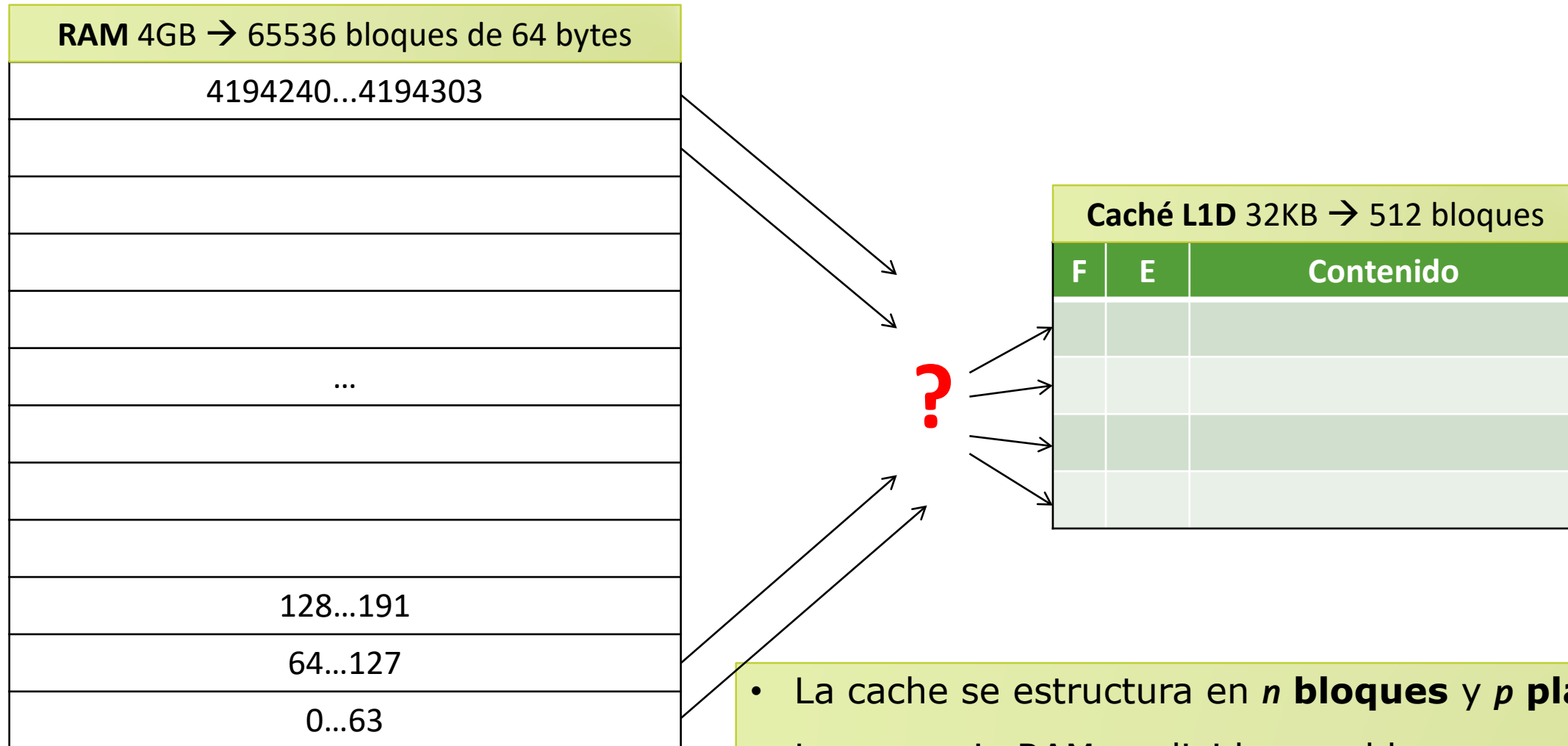
- **Bloque:** es la unidad mínima que se transfiere desde memoria RAM a caché y viceversa. Un bloque de 64 bytes contendría 16 palabras de 32 bits
- **Plano:** cada bloque puede tener uno o más planos según la configuración de la caché. El número de planos determina el **nivel de asociatividad** de la caché
- **Tamaño:** los bloques tienen un tamaño homogéneo. El tamaño de la caché es el producto del tamaño de bloque por el número de bloques y número de planos, p.e. 1024 bytes en el diagrama superior → 4 bloques X 4 planos X 64 bytes/bloque
- **Flags:** distintos indicadores que denotan el estado de cada bloque, según su tipo
- **Etiqueta:** identificador único de cada bloque que facilita el enlace entre memoria RAM y caché

Cachés L1 en procesadores actuales

Características

- **Tamaño de bloque (TB):** desde hace años el tamaño de bloque en la mayoría de procesadores es de 64 bytes, si bien algunos más recientes (como los Apple M3/M4) usan tamaños distintos
- **Número de bloques (NB):** lo usual es tener entre 64 y 192 bloques según la gama del procesador
- **Número de planos (NP):** indicado habitualmente como *associativity* en las hojas de especificaciones, suele ser de 4 u 8 para la L1 y algo superior para las L2/L3
- **Tamaño de plano (TP):** un plano engloba todos los bloques que están en un plano, por lo que su tamaño es $TB \times NB$.
P.e. $64 \text{ bloques} \times 64 \text{ bytes/bloque} = 4096 \text{ bytes/plano}$
- **Tamaño total:** en la actualidad los tamaños más habituales de las cachés L1I y L1D es de 32KB, 48KB o 64KB. En procesadores más antiguos eran corrientes tamaños de 4KB y 8KB, mientras que en los más modernos y de alto rendimiento se alcanzan los 192KB. Este tamaño se obtiene como $NP \times TP$, o bien como $TB \times NB \times NP$

Relación entre memoria RAM y caché



- La cache se estructura en n **bloques** y p **planos**
- La memoria RAM se divide en m bloques, con $m \gg n \times p$
- ¿Cómo se asocian los bloques de caché con los de RAM?

Tipos de caché y su funcionamiento



Terminología necesaria

Razonamiento

La memoria caché puede almacenar una fracción del contenido de la memoria principal, por lo que se precisa una estrategia que maximice el rendimiento y evite todos los accesos posibles a la RAM

Terminología

- **Acierto:** cuando la CPU precisa un dato y este está disponible en caché se tiene un acierto (*hit*), de forma que no es preciso acceder a memoria RAM
- **Fallo:** si el dato no está en caché y hay que acceder a RAM se tiene un fallo (*miss*), lo que implica sustituir un bloque de la caché, actualmente en uso, por el bloque de RAM que contenga ese dato
- **Estrategia de sustitución:** procedimiento por el que se decide qué bloque de la caché es sustituido por un bloque procedente de la memoria principal
- **Estrategia de escritura:** procedimiento que determina cómo se harán las escrituras en memoria

Tipos de memoria caché

Introducción

Según cómo se organicen los bloques y planos dispondremos de diferentes tipos de memoria caché, con un funcionamiento específico

Tipos y características

- **Correspondencia directa**

Se asigna a cada bloque de la RAM una correspondencia directa con uno de los bloques de la caché, en un solo plano. Ofrece la menor latencia, pero es poco flexible

- **Completamente asociativa**

Consiste en tener tantos planos como bloques permita el tamaño de la caché y realizar una búsqueda basándose en las etiquetas de los bloques. Ofrece la mayor flexibilidad, pero también tiene la mayor latencia

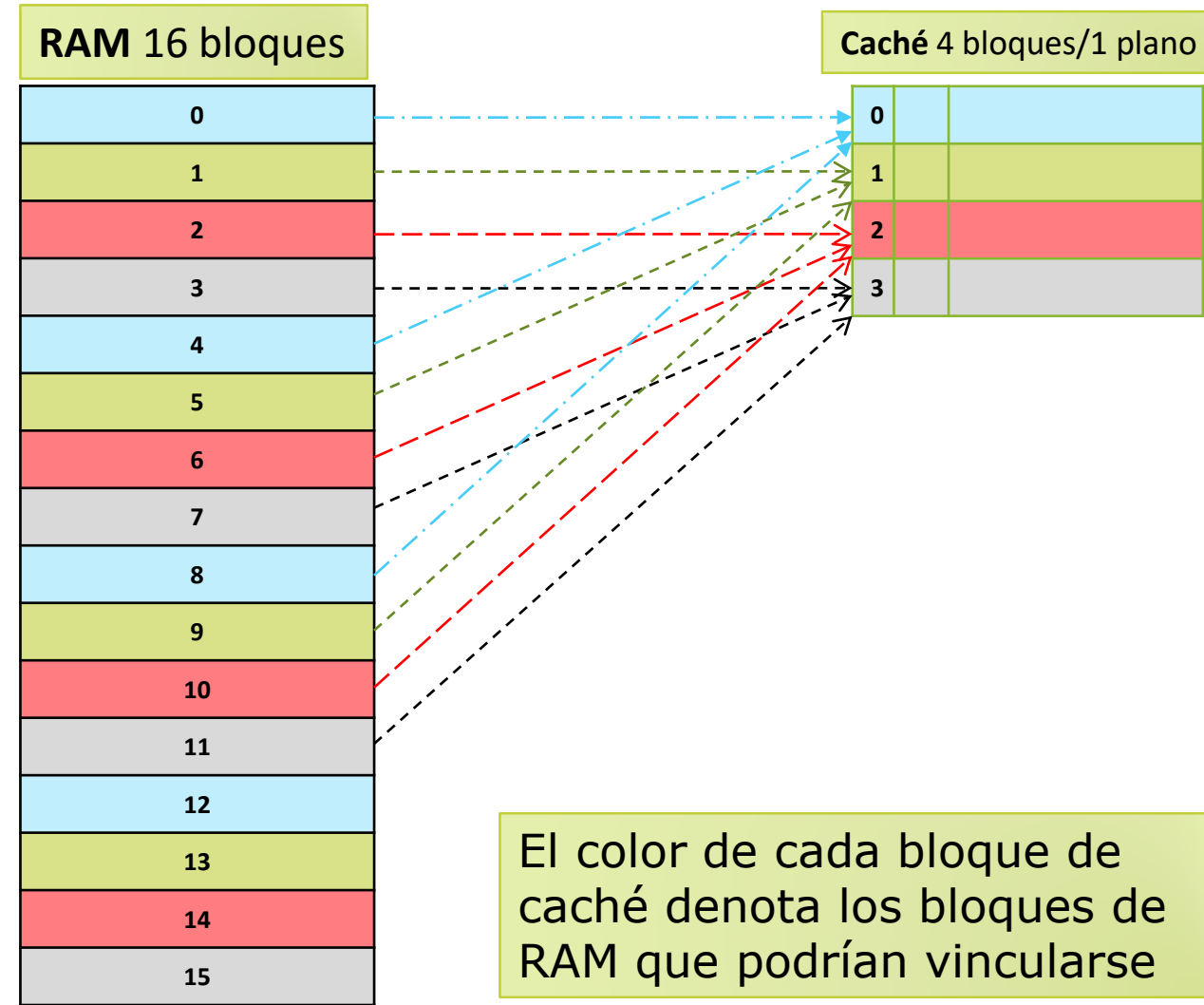
- **Asociativa por conjuntos**

Combina características de los dos tipos anteriores para alcanzar un equilibrio entre latencia y flexibilidad

Caché de correspondencia directa

Funcionamiento

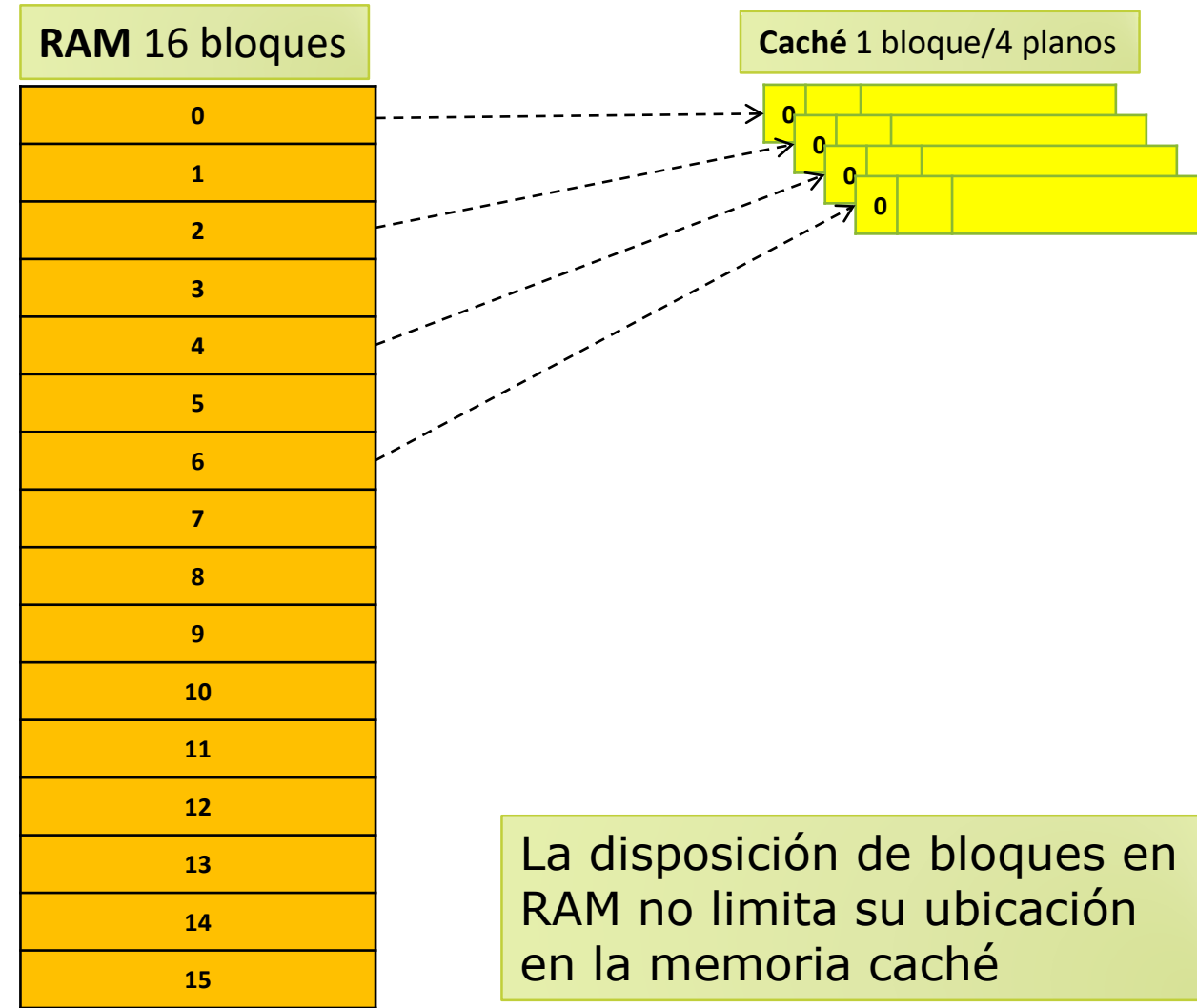
- La cache se estructura en **n bloques** y hay un único plano
- Cada bloque de RAM se asocia de **forma directa** con uno de caché según la fórmula $s = m \% n$, siendo % el módulo de división entera
- Encontrar el bloque de caché que corresponde a una cierta dirección de memoria es inmediato, la **latencia es mínima** (positivo)
- Si se accede a posiciones de memoria alojadas en bloques múltiplos de n , **habrá sustituciones continuas de entradas** de caché por nuevos bloques (negativo)
- Mecanismo de caché **sencillo de implementar** en hardware, pero muy **sensible a los patrones** de acceso a memoria por parte de los programas



Caché completamente asociativa

Funcionamiento

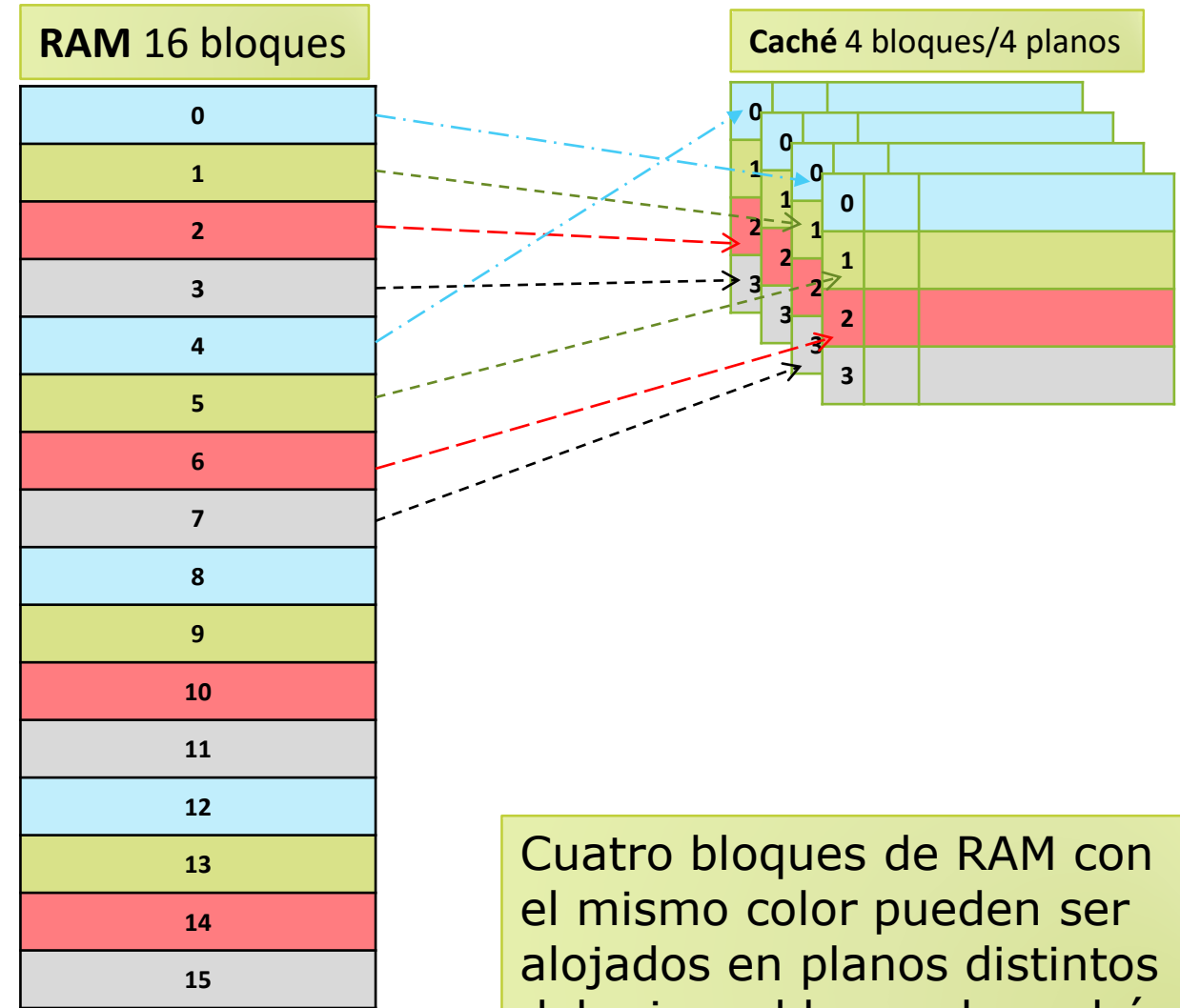
- La cache se estructura en n **planos** y hay un único bloque por plano
- **Cualquier bloque** de RAM se puede llevar a cualquiera de los planos del bloque de caché
- Determinar el plano donde está el bloque que corresponde a una dirección requiere múltiples comparaciones, la **latencia es mayor** (negativo)
- No **habrá sustituciones continuas de entradas** si se accede a varios bloques de RAM de forma repetitiva, sin que importe su disposición (positivo)
- Se requiere una **heurística de reemplazo** para determinar, cuando todos los planos de la caché están ocupados y hay que traer un nuevo bloque desde RAM, qué bloque es el que se descarta



Caché asociativa por conjuntos

Funcionamiento

- La cache se estructura en n **bloques** y p **planos**
- Cada bloque de RAM puede llevarse a **cualquiera de los planos** del bloque que le corresponda
- Encontrar el bloque de caché que corresponde a una cierta dirección de memoria conlleva **dos pasos**:
 - Determinar el bloque con la misma aritmética usada para la correspondencia directa
 - Buscar en los planos de ese bloque la marca
- La **latencia es mayor** que en la caché de correspondencia directa, pero **no tan alta** como en la completamente asociativa
- Un cierto número de bloques de RAM a los que correspondería el mismo bloque pueden coexistir en la caché, sin necesidad de reemplazo, lo que es **más flexible** que en correspondencia directa
- Aúna lo mejor** de los dos tipos de caché previos



Procedimiento para acceder a memoria

Necesidad

El acceso de la CPU a la memoria ya no es directo. Requiere unos pasos previos para determinar si el dato solicitado está disponible en caché o no

Algoritmo

1. Descomposición de la dirección

La dirección de memoria a la que se va a acceder se descompone en varias partes que determinan la **marca**, el **bloque**, la **palabra** dentro del bloque y el **byte** dentro de la palabra al que se quiere acceder

2. Comprobación de la marca

Con el número de bloque, se busca la marca entre los distintos planos del mismo. Si no se encuentra la marca, entonces se tiene **un fallo** y hay que traer el bloque desde RAM

3. Verificación de si la entrada es válida

Si se encuentra la marca, hay que determinar si el bloque contiene información válida mediante la verificación de sus indicadores. Si lo es, se tiene **un acierto**, en caso contrario, **un fallo**

Descomposición de la dirección

- **Direcciones de memoria**

Trabajamos con direcciones de memoria de 32 bits y palabras de 32 bits

- **Configuración de caché**

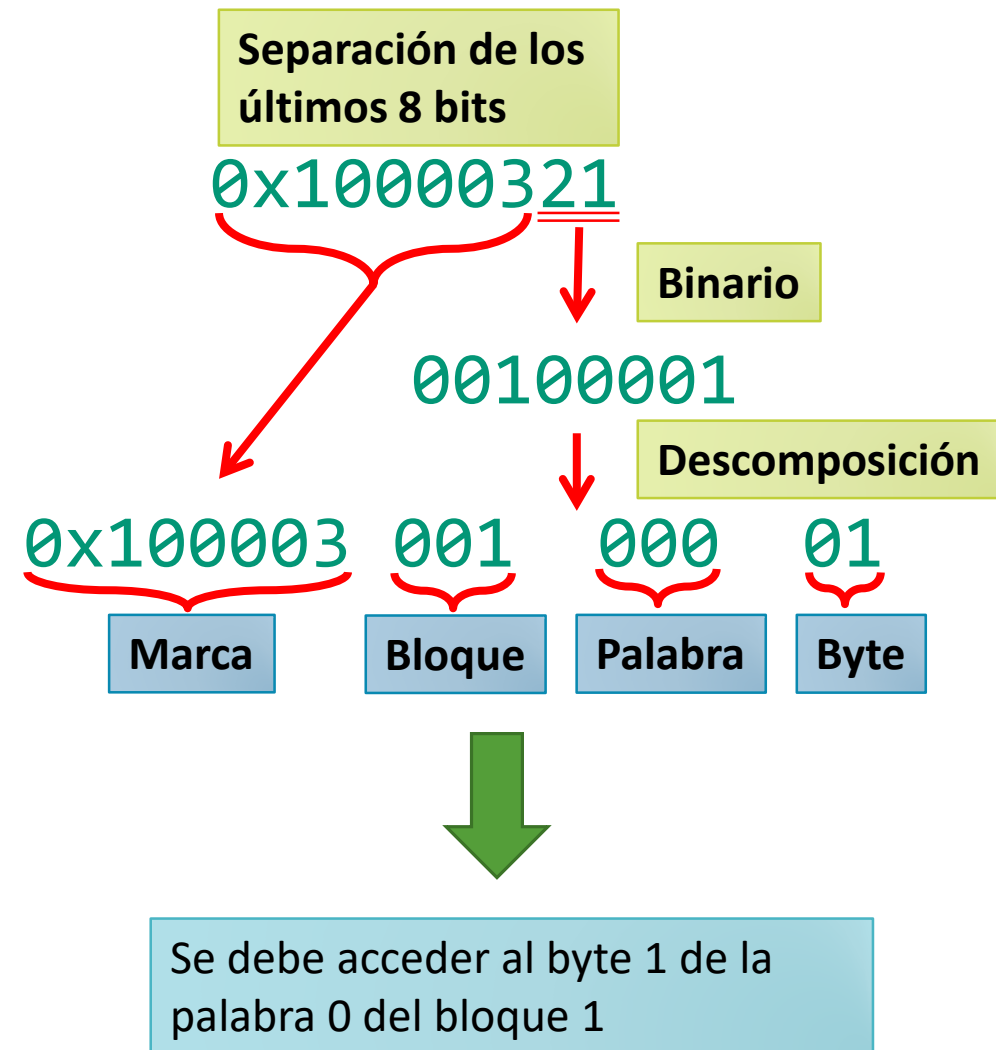
Se tiene una caché con 2 planos de 8 bloques cada uno y un tamaño de bloque de 32 bytes (8 palabras de 32 bits). Por tanto se precisan:

- **3 bits** para indicar el **bloque**
- **3 bits** para denotar la **palabra** dentro del bloque
- **2 bits** para seleccionar un **byte** de la palabra

Los restantes **24 bits** serán la **marca** que identificará cada bloque único de la RAM

- **Operación a ejecutar**

La CPU quiere leer el contenido de la dirección de memoria 0x10000321



Comprobación de la marca

- **Planos del bloque 1**

Se recorren los planos del bloque que corresponda, comparando su marca con la de la dirección

- **Acierto (provisional)**

Si se encuentra la marca tenemos un acierto, pero aun hay que verificar la validez del bloque

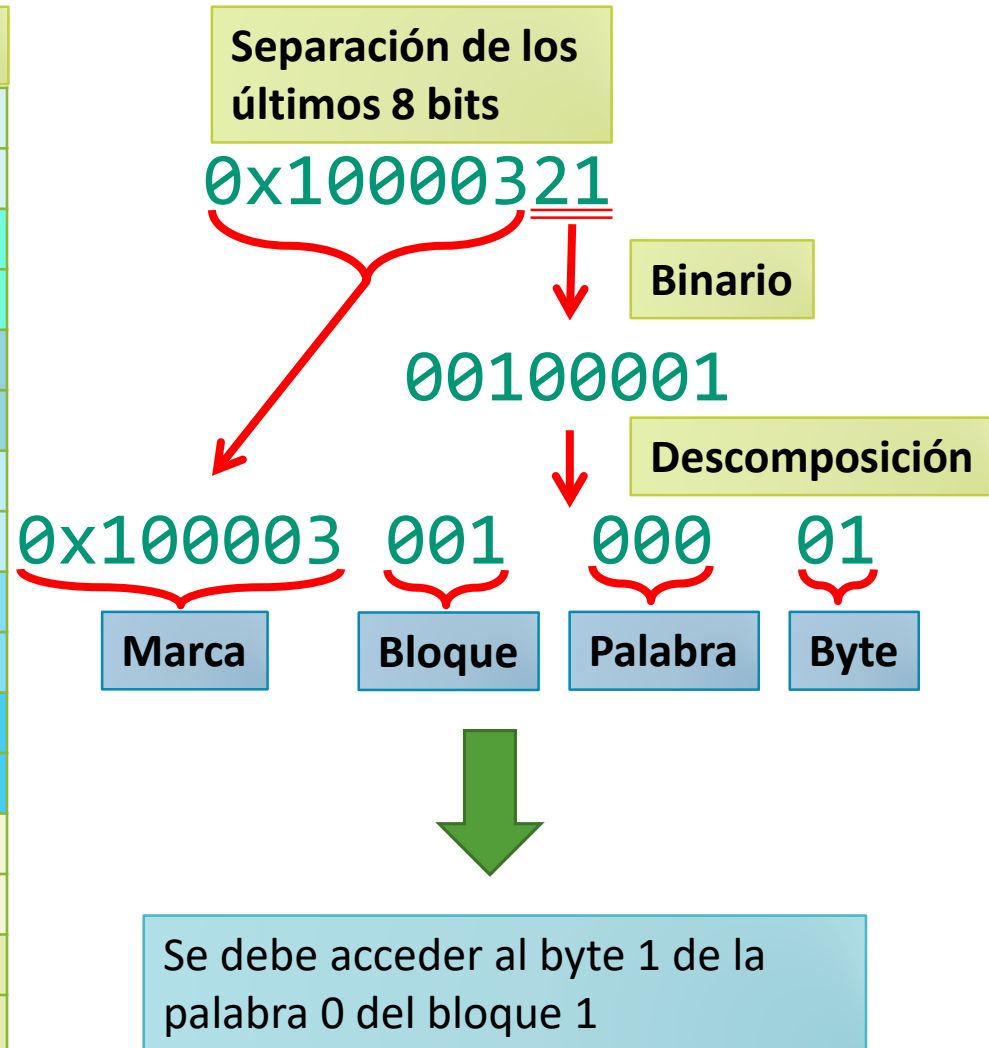
- **Fallo**

Si ninguno de los planos del bloque contiene la marca se produce un fallo

- **Sustitución**

Hay que traer desde RAM el bloque que corresponde a la dirección 0x10000321

8 bloques X 2 planos		
0		
0		
1	0x100006	
1	0x100003	
2		
2		
3		
3		
4		
4		
5		
5		
6		
6		
7		
7		



Verificación de si la entrada es válida

- **Indicadores**

Cada bloque de la caché tiene un campo de *flags* o indicadores que denotan su estado

- **Bit V**

Uno de los indicadores señalará si el contenido del bloque en caché contiene datos válidos o no. Si este bit no está activo se tendrá un fallo, en caso contrario un acierto

- **Validez**

Un bloque de caché no contendrá datos válidos si nunca se ha copiado en él un bloque desde RAM, lo cual sucede al inicio de la ejecución de un programa

- **Coherencia**

En sistemas con múltiples CPU, una entrada de caché en un núcleo puede ser invalidada si tiene un bloque compartido con otro núcleo desde el que se ha modificado su contenido

I	Flags	Marca	Datos
0			
0			
1		0x100006	
1	V	0x100003	
2			
2			
3			
3			
4			
4			
5			
5			
6			
6			
7			
7			

Estrategia de reemplazo

Necesidad

Cuando se precisa un dato que no está en caché, se trae desde memoria RAM un bloque completo que sustituye a uno de los existentes en caché

Estrategias

- **Directa**
En una caché de correspondencia directa siempre se sustituye el bloque que corresponda (según la dirección de acceso) con el nuevo contenido traído desde RAM
- **Aleatoria**
Si se tienen varios planos en el bloque, se toma uno de ellos de forma aleatoria para copiar en él el nuevo bloque leído desde memoria RAM
- **LRU (*Least Recently Used*)**
Cada bloque/plano tiene asociado un índice que denota cuándo fue accedida por última vez la entrada en caché, de forma que se sustituya aquella que hace más tiempo que no se ha usado
- **Otras**
Existen otras estrategias como FIFO (*First In First Out*), que sustituye el bloque más antiguo de la caché, y LFU (*Least Frequently Used*), que sustituye el que se usa con menor frecuencia

Estrategia de escritura

Necesidad

Al escribir un dato en memoria es preciso determinar cómo se preservará la coherencia entre los diferentes niveles de caché y la RAM

Estrategias

- **Inmediata (*write through*)**

Cualquier escritura en memoria se efectúa tanto en caché como en RAM, de forma que se mantiene la coherencia en todo momento

- **Demorada (*write back*)**

Las escrituras se realizan en caché, marcando el bloque con el indicador *Dirty*. Este denota que si el bloque se sustituye por otro, antes hay que llevar los cambios a RAM

- **Sustitución en escritura (*write allocate*)**

Si la escritura afecta a una dirección que no está en caché, el bloque puede ser traído desde RAM a caché antes (*write allocate*) o bien escribir directamente en RAM sin esa sustitución (*no write allocate*)

Análisis del impacto de la caché en el rendimiento



Tasas de aciertos y fallos

Razonamiento

Al acceder a un dato desde la CPU, la dirección puede corresponder a un bloque válido que esté en caché (acierto) o a uno que haya que traer desde RAM (fallo)

Formulación

- **Nº de aciertos (N_a):** es el conteo del número de accesos a memoria que son satisfechos desde la caché
- **Nº de fallos (N_f):** es el conteo del número de accesos a memoria que precisan el reemplazo de un bloque en caché
- **Nº total de accesos (N_t):** se calcula como $N_t = N_a + N_f$
- **Ratio de aciertos:** se calcula como $R_a = N_a / N_t$ y se expresa como porcentaje
- **Ratio de fallos:** se calcula como $R_f = N_f / N_t$ y se expresa como porcentaje
- **Objetivo:** maximizar la ratio de aciertos, lo cual implica minimizar la de fallos

Tiempos de acceso

Razonamiento

El acceso a un dato en caché requiere un tiempo muy inferior al del acceso a memoria principal, algo que es preciso medir de forma objetiva

Formulación

- **Tiempo de acceso a caché (T_a):** es el tiempo que se tarda en obtener una palabra desde un bloque en memoria caché
- **Tiempo de acceso a RAM (T_f):** es el tiempo empleado en traer un bloque desde RAM a la ubicación que corresponda de la caché y obtener la palabra desde caché
- **Tiempo total:** se calcula como $T_t = N_a \times T_a + N_f \times T_f$
- **Tiempo de acceso medio:** se calcula como $T_m = \frac{T_t}{N_t} = R_a \times T_a + R_f \times T_f$
- **Objetivo:** si se reduce T_m se mejorará el rendimiento al ejecutar un programa, con independencia de cuántos accesos se efectúen en total a memoria

Cómo maximizar la ratio de aciertos

Razonamiento

Mejorar la tasa de aciertos implica reducir proporcionalmente el tiempo de acceso medio a memoria y, por tanto, mejorar el rendimiento, por lo que nos interesa conocer los factores que afectarían a esa ratio

Factores a considerar

- **Tamaño de la caché:** en general, la ratio de aciertos se verá incrementada a medida que también lo haga la cantidad de caché disponible
- **Configuración de la caché:** el tamaño de bloque, número de bloques, nivel de asociatividad, estrategia de reemplazo y de escritura tienen también una influencia significativa en la ratio de aciertos
- **Patrones de acceso a memoria:** asimismo, los patrones que sigan los programas al hacer referencia a posiciones de memoria determinarán en buena medida que haya más aciertos o fallos, con independencia de los dos factores previos

Tamaño de la caché

Espacio físico

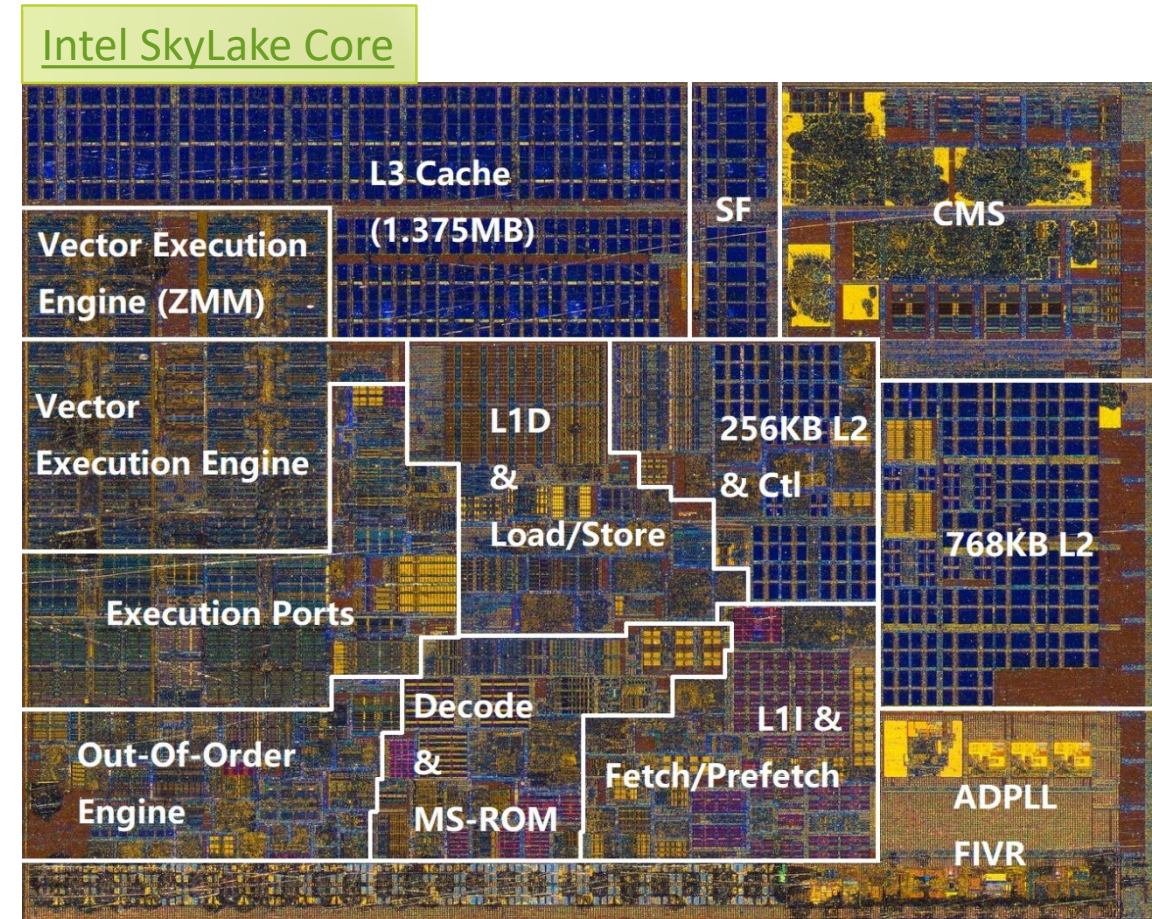
La tecnología SRAM con la que se fabrica la memoria caché ocupa mucho más espacio que la DRAM, lo que implica sacrificar un área para esta tarea que podría servir para incluir más unidades de ejecución u otros elementos

Coste económico

La fabricación de SRAM resulta mucho más cara que la de memoria DRAM estándar, estimándose un factor de 100 a 1 por unidad de memoria

Tecnología actual

Las limitaciones de espacio y coste se deben a la tecnología con la que actualmente se produce la memoria caché, algo que podría cambiar en el futuro y que permitiría contar con más memoria caché y, por tanto, incrementar la ratio de aciertos



Configuración de la caché

Tamaño de bloque

Cuanto mayor es un bloque de caché menor es el número de fallos en el acceso a direcciones consecutivas de memoria, pero se incrementa el tiempo preciso para traer un bloque desde RAM

Nivel de asociatividad

Una memoria asociativa por conjuntos puede llegar a ser tan efectiva como una completamente asociativa, pero con una menor latencia de búsqueda

Estrategia de reemplazo

En general, técnicas de reemplazo como LRU ofrecen una mejor ratio de aciertos que otras más básicas como la aleatoria

Estrategia de escritura

La escritura demorada, con sustitución de bloques que no estén ya en caché, suele tener un mejor rendimiento que otras opciones

Dependencias

En último extremo muchos de estos aspectos dependerán del patrón de accesos a memoria que presente el programa a ejecutar

Patrones de acceso a memoria

Principio de localidad espacial

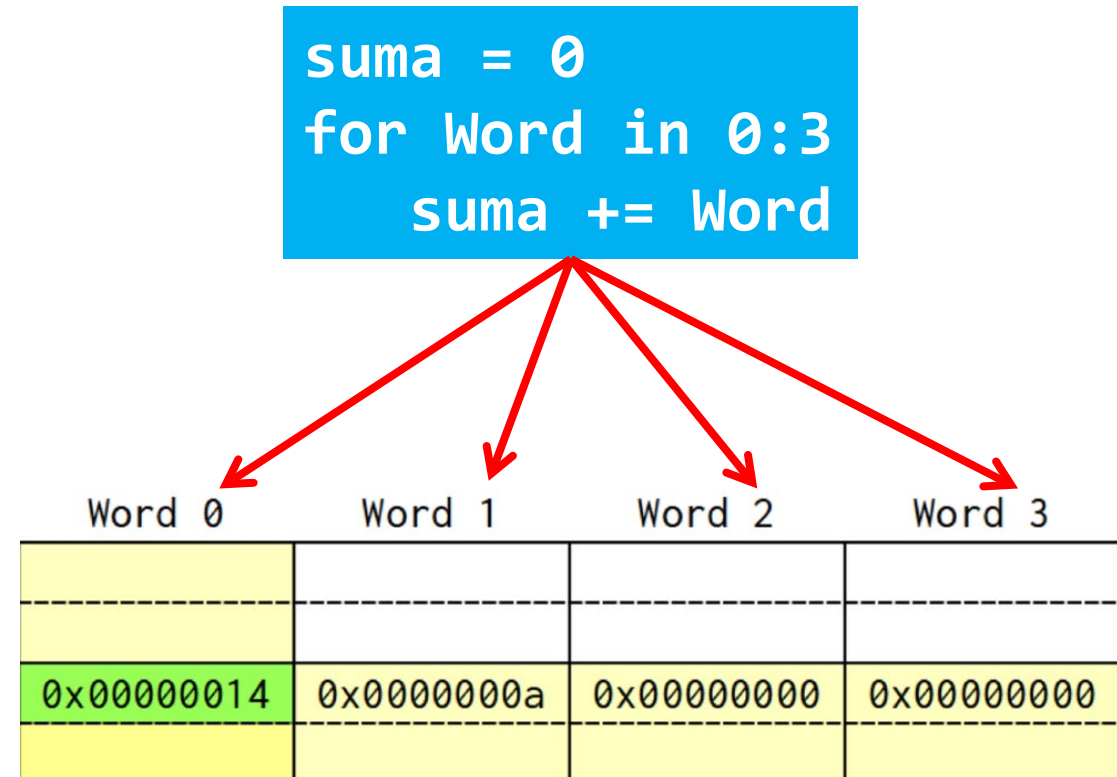
La mayoría de programas acceden a datos que se alojan en posiciones consecutivas de memoria, por lo que una vez que un bloque está en caché el número de aciertos será mayor

Principio de localidad temporal

De forma análoga, es normal que en unidades de tiempo consecutivas se use un mismo dato, como ocurre con la variable suma de la derecha, lo que también contribuye a mejorar la ratio de aciertos

Análisis de los patrones de acceso

Analizar la ejecución del código, para identificar los patrones de acceso y verificar que se respeten los dos principios anteriores, es preciso para garantizar un buen uso de la memoria caché



Cuestiones clave de este tema



Cuestiones clave

Qué deberías saber

*Al inicio de este tema se planteaban unos objetivos específicos que deberían permitirte **responder a las siguientes cuestiones clave***

Cuestiones

- ¿Qué finalidad tiene la memoria caché en la arquitectura de un computador?
- ¿Cuál es el impacto en el rendimiento que tiene la memoria caché?
- ¿Cuáles son los diferentes niveles de caché existentes?
- ¿En qué se diferencian los tipos de caché según su asociatividad?
- ¿Cómo operan las estrategias de reemplazo y escritura básicas?
- ¿Cómo se calculan las ratios de acierto y fallos para evaluar las mejoras?
- ¿Qué impacto tienen los patrones de acceso a memoria en la caché?

Material adicional

Descripción

Para ampliar tus conocimientos sobre los contenidos de este tema te recomendamos que consultes los recursos indicados a continuación.

Recursos

- **Capítulo 4** (*Memoria caché*) del libro **Organización y arquitectura de computadores 7ED** disponible en [formato digital](#) en la BUJA (recuerda identificarte para poder acceder a leerlo desde tu navegador)
- **Capítulo 2** (*Memory Hierarchy Design*) del libro **Computer Architecture 5ED** disponible en [formato digital](#) en la BUJA (recuerda identificarte para poder acceder a leerlo desde tu navegador)
- **Capítulo 5** (*Large and Fast: Exploiting Memory Hierarchy*) del libro **Computer Organization and Design** disponible en la Biblioteca de la UJA ([enlace](#))